Programming C – Arrays

What is an Array?

- An **array** is a collection of elements of the **same type** stored in contiguous memory locations.
- It allows storing multiple values under a single name.

Example:

```
int numbers[5];
```

- This creates space for 5 integers.

Why Use Arrays?

Without arrays:

```
int n1 = 10, n2 = 20, n3 = 30, n4 = 40, n5 = 50;
```

With arrays:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Advantages

PROF

- Easier to manage many values
- Simplifies loops and data processing
- Improves code organization

Array Declaration

Syntax:

```
type array_name[size];
```

Examples:

```
```

```c
int marks[10];
float prices[5];
char letters[26];
```

- `size` must be a constant expression

Array Initialization

At declaration

```c
int numbers[5] = {1, 2, 3, 4, 5};
```

Partial initialization

```c
int numbers[5] = {1, 2};
```

Remaining elements → initialized to 0

Without size

```c
int numbers[] = {1, 2, 3};
```

Accessing Elements

PROF

- Use index (starting from 0)

```c
int numbers[3] = {10, 20, 30};

printf("%d", numbers[0]); // 10
printf("%d", numbers[1]); // 20
```

Important

- Index range: `0` to `size - 1`

Modifying Elements

```c
int numbers[3] = {10, 20, 30};

numbers[1] = 50;
```

Now array becomes:

```
{10, 50, 30}
```

Arrays and Loops

- Arrays are commonly used with loops

```c
int numbers[5] = {1, 2, 3, 4, 5};

for (int i = 0; i < 5; i++) {
    printf("%d ", numbers[i]);
}
```

Reading Values into Arrays

```c
int numbers[3];

for (int i = 0; i < 3; i++) {
    scanf("%d", &numbers[i]);
}
```


- Each element is filled individually

Example: Sum of Array Elements

```c
int numbers[5] = {1, 2, 3, 4, 5};
int sum = 0;

for (int i = 0; i < 5; i++) {
    sum += numbers[i];
}

printf("Sum = %d", sum);
```

Example: Find Maximum

```
int numbers[5] = {3, 7, 2, 9, 5};
int max = numbers[0];

for (int i = 1; i < 5; i++) {
    if (numbers[i] > max) {
        max = numbers[i];
    }
}

printf("Max = %d", max);
```

Multidimensional Arrays

- Arrays with more than one dimension

Example: 2D array (matrix)

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

- 2 rows, 3 columns

Accessing 2D Arrays

PROF

```
printf("%d", matrix[0][1]); // 2
```

- First index → row
- Second index → column

Looping Through 2D Arrays

```
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", matrix[i][j]);
    }
```

```
    printf("\n");
}
```

Arrays of Characters (Strings Intro)

```
char name[6] = "Hello";
```

- Strings end with special (null) character `'\0'` (whose value is 0)

Example, the string literal constant "Hello" is actually stored as:

```
'H' 'e' 'l' 'l' 'o' '\0'
```

- If an array of char does not contain any `\0` it should not be considered a string

---

Reading Strings

```
char name[20];
scanf("%s", name);
```

- Reads until whitespace
- Automatically inserts a final null character (if there is room in the array)
- If we want to read till the end-of-line or limit the characters to read better to use other functions such as `fgets`

PROF

Common Mistakes

- Accessing out-of-bounds index:

```
numbers[5]; // invalid if size is 5
```

- Forgetting array size
- Using uninitialized arrays
- An array of chars may not be a string (if it does not contain any '\0')

Fixed Size Limitation

- Array size must be known at compile time (in basic C)
- Cannot resize during execution
- Recent versions of C compiler admit Variable Length Arrays (VLA): the arrays size can be an expression (cannot change)

Passing Arrays to Functions

```c
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}
```

Usage:

```c
int numbers[3] = {1, 2, 3};
printArray(numbers, 3);
```

Modifying Array Elements in a Function

 Arrays are passed in a way that allows modification of original data

```c
void doubleElements(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = arr[i] * 2;
    }
}
```

PROF

Usage:

```c
int numbers[3] = {1, 2, 3};

doubleElements(numbers, 3);

// numbers becomes: {2, 4, 6}
```

 Key idea:

- The function works on the **same array**, not a copy

Key Observations

- Arrays store multiple values of same type
- Index starts at 0
- Size is fixed
- Often used with loops
- Passed to functions "by reference" (array values can be modified)

Practice Exercises

1. Create an array of 5 integers and print them
2. Compute the average of elements
3. Reverse an array
4. Find the minimum value
5. Work with a 2D matrix (sum all elements)

Programming C – Pointers (and their relationship with Arrays)

What is a Pointer?

- A **pointer** stores the **memory address** of another variable

```
int x = 10;
int *p = &x;
```

- `p` → address of `x`
- `*p` → value of `x`

Memory and Addresses

```
int x = 10;
printf("%p", &x);
```

- `&x` gives address
- Memory = sequence of addressed locations

Declaring and Initializing Pointers

PROF

```
int *p;
int x = 10;
p = &x;
```

⚠ Avoid:

```
int *p; // uninitialized
```

Dereferencing

```
int x = 10;
int *p = &x;
```

```
printf("%d", *p);
```

- `*p` accesses value at stored address

NULL and Safety

```
int *p = NULL;
```

- Always check before dereferencing

Modifying Variables via Pointers

```
int x = 10;
int *p = &x;

*p = 50;
```

- Now `x = 50`
- currently `*p` is an alias of `x`

Passing Pointers to Functions

```
void change(int *x, int k) {
    *x += k;
}
```

PROF

Usage:

```
change(&a, -5);
```

- note the use of `&` operator in the function call (like in scanf)
- effect: the value of variable `a` is decreased by five

Careful (never use conventional data values as addresses)

```
change(100, -5);
```

Modifies memory at location 100 (unsafe)

Use pointers to emulate multiple return values

```
#include <stdio.h>

// Function that computes sum and product
// and returns them via pointer parameters
void compute(int a, int b, int *sum, int *product) {
    *sum = a + b;
    *product = a * b;
}

int main() {
    int x = 4, y = 5;
    int s, p;

    compute(x, y, &s, &p);

    printf("Sum = %d\n", s);
    printf("Product = %d\n", p);

    return 0;
}
```

Why Pointers After Arrays?

PROF

```
int numbers[3] = {10, 20, 30};
```

- Questions:
    - Where are elements stored?
    - How are they accessed efficiently?

□ Pointers provide direct access to memory and explain array behavior

---

Arrays and Pointers Connection

```
int arr[3] = {10, 20, 30};
```

- Array name acts like address of first element

```
arr[1] == *(arr + 1)
```

Pointer Arithmetic (only in the context of an array!)

```
int *p = arr;

p++;
```

- Moves (points) to next int element in the memory
- For example, if the address `arr` is `1000`, after `p++` the value of `p` becomes `1004`

```
printf("%d", *p); // 20
```

Traversing Arrays with Pointers

```
for (int i = 0; i < 3; i++) {
    printf("%d ", *(arr + i));
}
```

Passing Arrays to Functions

PROF

```
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}
```

2D Arrays Recap

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

- Access:

```
matrix[1][2]; // 6
```

Passing 2D Arrays to Functions

- Must specify column size

```
void printMatrix(int m[][3], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }
}
```

Usage:

```
printMatrix(matrix, 2);
```

Why Column Size is Required

- Compiler needs column size to compute positions:


```
m[i][j] → *(m[i] + j)
```

- Without column size → cannot locate elements

Pointer Representation of 2D Arrays

- A 2D array is stored **row by row**

```
int matrix[2][3] = {{1, 2, 3},
                    {4, 5, 6}
                   };
```

Memory layout:

```
1 2 3 4 5 6
```

Using Pointers with 2D Arrays

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

int (*p)[3] = matrix;
```

- `p` points to a row (array of 3 integers)

Accessing Elements with Pointer to 2D Array

```
printf("%d", p[1][2]);     // 6
printf("%d", *(*(p+1)+2)); // same
```

Explanation:

- `p+1` → next row
- `*(p+1)` → row
- `+2` → column

PROF

Traversing 2D Array with Pointers

```
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", *(*(p + i) + j));
    }
    printf("\n");
}
```

Function with Pointer to 2D Array

```c
void printMatrixPtr(int (*m)[3], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", *(*(m + i) + j));
        }
        printf("\n");
    }
}
```

Alternative: Flattened Access

```c
int *p = &matrix[0][0];

for (int i = 0; i < 6; i++) {
    printf("%d ", *(p + i));
}
```

- Treats matrix as linear memory

Common Errors

- Wrong pointer type for 2D arrays
- Forgetting column size in functions
- Out-of-bounds access
- Misusing pointer arithmetic

Summary

PROF

- Pointers give access to memory
- Arrays and pointers are closely related
- 2D arrays:
    - Require column size in functions
    - Can be accessed via pointer arithmetic
- Pointer-to-array enables structured access

Practice Exercises

1. Write a function to print a 2D array
2. Use pointer notation instead of indexing

3. Sum all elements of a matrix using pointers
4. Access elements using flattened memory
5. Pass a matrix to a function and modify values

Final Insight

 Arrays organize data
 Pointers reveal and control **how data is stored and accessed in memory**, even in multidimensional structures

---